

SECTION 1: ENGLISH (EN)

Architectural Checklist 2026

Technical blueprint for next-generation web engineering.

01. Next.js App Router & SSR Optimization

■ Partial Prerendering (PPR)

Combine static shell with dynamic holes for instant loading:

```
// next.config.js
const nextConfig = { experimental: { ppr: 'incremental' } };

// page.tsx
import { Suspense } from 'react';
export default function Page() {
  return (
    <main>
      <StaticHeader />
      <Suspense fallback=<Skeleton />>
        <DynamicContent />
      </Suspense>
    </main>
  );
}
```

■ Server Actions Security

Implement strict mutation barriers including Zod validation and rate limiting:

```
export async function updateProfile(formData: FormData) {
  const session = await auth();
  if (!session) throw new Error("Unauthorized");

  await rateLimit(session.user.id);
  const data = schema.safeParse(Object.fromEntries(formData));
  if (!data.success) return { error: "Invalid payload" };
  // ... db update
}
```

■ Route Segment Configuration

Use granular caching semantics per route segment:

```
// Cache for 1 hour vs Force dynamic
export const revalidate = 3600;
export const dynamic = 'force-dynamic';
```

02. AI & LLM Dialog Agents Integration

■ Latency Optimization via Streaming

Stream tokens to the client directly via Vercel AI SDK:

```
import { streamText } from 'ai';
import { openai } from '@ai-sdk/openai';

export async function POST(req: Request) {
  const { prompt } = await req.json();
  const result = await streamText({
    model: openai('gpt-4-turbo'),
    prompt,
  });
  return result.toDataStreamResponse();
}
```

■ Vector Cache & Retry Policy

Cache embeddings in Redis/Valkey before querying Pinecone to reduce API costs. Implement an exponential backoff retry strategy for handling 429/503 HTTP errors.

03. Sub-100ms LCP on Edge CDN

■ Edge Runtime Migration

Move mission-critical middleware and fast API routes to the Edge:

```
// Opt into Edge runtime (No Node.js APIs)
export const runtime = 'edge';
export const preferredRegion = 'fra1'; // Frankfurt
```

■ Advanced Asset Optimization

Configure Next.js image optimization for AVIF and aggressive font caching:

```
// next.config.js
module.exports = {
  images: { formats: ['image/avif', 'image/webp'] },
};
```

■ Streaming HTML via Suspense

Separate heavy SQL queries from the initial HTML document response:

```
import { Suspense } from 'react';
<Suspense fallback=<Spinner />>
  <HeavyDatabaseComponent />
</Suspense>
```

04. EU GDPR Compliance & PII Encryption

■ Zero-Knowledge Architecture

Encrypt PII on-the-fly before Postgres insertion using Node crypto:

```
import crypto from 'crypto';
const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
let encrypted = cipher.update(piiData, 'utf8', 'hex');
encrypted += cipher.final('hex');
```

■ Consent Management

Load tracking scripts via Partytown or worker strategy to prevent blocking:

```
import Script from 'next/script';
<Script src="gtm.js" strategy="worker" />
```

SECTION 2: SRPSKI / CRNOGORSKI (SRB/CNR)

Arhitektonski ček-list 2026

Tehnički nacrt za sledeću generaciju veb inženjeringa.

01. Next.js App Router i SSR optimizacija

■ Partial Prerendering (PPR)

Kombinujte statički kostur sa dinamičkim delovima za trenutno učitavanje:

```
// next.config.js
const nextConfig = { experimental: { ppr: 'incremental' } };

// page.tsx
import { Suspense } from 'react';
export default function Page() {
  return (
    <main>
      <StaticHeader />
      <Suspense fallback=<Skeleton />>
        <DynamicContent />
      </Suspense>
    </main>
  );
}
```

■ Server Actions Security

Implementirajte striktne barijere, Zod validaciju i limitiranje zahteva:

```
export async function updateProfile(formData: FormData) {
  const session = await auth();
  if (!session) throw new Error("Unauthorized");

  await rateLimit(session.user.id);
  const data = schema.safeParse(Object.fromEntries(formData));
  if (!data.success) return { error: "Invalid payload" };
  // ... db update
}
```

■ Route Segment Configuration

Koristite granularno keširanje po ruti:

```
// Keš na 1 sat naspram forsiranja dinamičkog renderovanja
export const revalidate = 3600;
export const dynamic = 'force-dynamic';
```

02. Integracija AI i LLM agenata

■ Latency Optimization (Strimovanje)

Strimujte tokene direktno klijentu putem Vercel AI SDK-a:

```
import { streamText } from 'ai';
import { openai } from '@ai-sdk/openai';

export async function POST(req: Request) {
  const { prompt } = await req.json();
  const result = await streamText({
    model: openai('gpt-4-turbo'),
    prompt,
  });
  return result.toDataStreamResponse();
}
```

■ Vector Cache i Retry Policy

Keširajte embedding podatke u Redis/Valkey bazama. Implementirajte eksponencijalni backoff za rešavanje grešaka (429/503).

03. Sub-100ms LCP na Edge CDN-u

■ Edge Runtime

Prebacite kritični middleware i brze API rute na Edge:

```
// Prelazak na Edge runtime (Bez Node.js API-ja)
export const runtime = 'edge';
export const preferredRegion = 'fra1'; // Frankfurt
```

■ Advanced Asset Optimization

Konfigurirajte Next.js optimizaciju slika za AVIF i agresivno keširanje fontova:

```
// next.config.js
module.exports = {
  images: { formats: ['image/avif', 'image/webp'] },
};
```

■ Streaming HTML (Suspense)

Odvojte teške SQL upite od početnog HTML odgovora:

```
import { Suspense } from 'react';
<Suspense fallback=<Spinner />>
  <HeavyDatabaseComponent />
</Suspense>
```

04. Usklađenost sa EU GDPR i PII enkripcija

■ Zero-Knowledge Architecture

Enkriptujte PII pre unosa u bazu (Postgres) koristeći Node crypto:

```
import crypto from 'crypto';
const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
let encrypted = cipher.update(plaintext, 'utf8', 'hex');
encrypted += cipher.final('hex');
```

Učitavajte skripte za praćenje preko Partytown-a ili worker strategije:

```
import Script from 'next/script';
<Script src="/gtm.js" strategy="worker" />
```

Архитектурный чек-лист 2026

Технический чертёж для веб-инженерии следующего поколения.

01. Оптимизация Next.js App Router & SSR

■ Partial Prerendering (PPR)

Комбинация статического скелета и динамических островов для мгновенной загрузки:

```
// next.config.js
const nextConfig = { experimental: { ppr: 'incremental' } };

// page.tsx
import { Suspense } from 'react';
export default function Page() {
  return (
    <main>
      <StaticHeader />
      <Suspense fallback=<Skeleton />>
        <DynamicContent />
      </Suspense>
    </main>
  );
}
```

■ Server Actions Security

Барьеры безопасности мутаций: Zod валидация, проверка сессии, rate limit:

```
export async function updateProfile(formData: FormData) {
  const session = await auth();
  if (!session) throw new Error("Unauthorized");

  await rateLimit(session.user.id);
  const data = schema.safeParse(Object.fromEntries(formData));
  if (!data.success) return { error: "Invalid payload" };
  // ... db update
}
```

■ Route Segment Configuration

Настройка семантики кэширования для отдельных сегментов:

```
// Кэширование на час или строго динамический роут
export const revalidate = 3600;
export const dynamic = 'force-dynamic';
```

02. Интеграция ИИ и Диалоговых агентов (AI & LLM)

■ Latency Optimization

Потоковая передача токенов клиенту через Vercel AI SDK:

```
import { streamText } from 'ai';
import { openai } from '@ai-sdk/openai';

export async function POST(req: Request) {
  const { prompt } = await req.json();
  const result = await streamText({
    model: openai('gpt-4-turbo'),
    prompt,
  });
  return result.toDataStreamResponse();
}
```

■ Vector Cache & Retry Policy

Локальное кэширование эмбедингов в Redis/Valkey перед Pinescone. Реализация паттерна экспоненциального бэк-оффа (retry-policy) при ошибках 429/503.

03. Загрузка Sub-100ms LCP на Edge CDN

■ Edge Runtime

Перенос middleware и быстрых API на Edge (ближе к пользователю):

```
// Использование Edge runtime (отсутствуют API Node.js)
export const runtime = 'edge';
export const preferredRegion = 'fra1'; // Frankfurt
```

■ Advanced Asset Optimization

Оптимизация изображений в AVIF и настройка кэширования шрифтов:

```
// next.config.js
module.exports = {
  images: { formats: ['image/avif', 'image/webp'] },
};
```

■ Streaming HTML

Отделение тяжелых SQL-запросов от мгновенного ответа сервера:

```
import { Suspense } from 'react';
<Suspense fallback={<Spinner />}>
  <HeavyDatabaseComponent />
</Suspense>
```

04. Соответствие EU GDPR и Шифрование данных

■ Zero-Knowledge Architecture

Шифрование "на лету" персональных данных перед записью в БД:

```
import crypto from 'crypto';
const cipher = crypto.createCipheriv('aes-256-gcm', key, iv);
let encrypted = cipher.update(piiData, 'utf8', 'hex');
encrypted += cipher.final('hex');
```

■ Consent Management

Изоляция GTM и аналитики для предотвращения блокировки UI потока:

```
import Script from 'next/script';
<Script src="gtm.js" strategy="worker" />
```